

Module 15: Optional

`Optional` exists for one job: **a return type for a method that may legitimately find nothing**. Used for that, it is excellent. Used anywhere else it is usually worse than the null it replaced, because it adds ceremony without adding safety.

This module is mostly about the difference.

What You'll Learn

- What `Optional` actually buys you, and who it buys it for
- The operations that never ask "is it there"
- `map` versus `flatMap`, the same rule as streams
- Why `orElse` and `orElseGet` are not interchangeable
- Five anti-patterns, including the one that reintroduces the crash

Coming From Another Language

	Elsewhere	Java
Absent value	<code>None</code> , <code>null</code> , <code>undefined</code>	<code>Optional.empty()</code>
Safe navigation	<code>?.</code>	<code>.map(...)</code>
Default	<code>x ?? y</code> , <code>or</code>	<code>.orElse(y)</code>
Compiler-enforced	Rust <code>Option</code> , Kotlin <code>T?</code>	no

That last row matters. In Rust or Kotlin the compiler stops you ignoring absence. Java's `Optional` is an ordinary class, and nothing prevents `.get()`. It is a **convention supported by a type**, not a guarantee.

The Lesson

What it is for

Compare two signatures:

```
User findUser(int id)           // might return null. Read the source to find out.
Optional<User> findUser(int id)  // says so, in the type.
```

That is the entire value proposition. It is about **the caller's obligation being visible**, not about avoiding null internally.

`Optional.of` rejects null on purpose. Writing it asserts the value is there, so an NPE means your assertion was wrong. Use `ofNullable` when you genuinely do not know.

Using it without unwrapping

The idiomatic style never asks whether a value is present. It describes what to do with it and what to fall back to. From `OptionalBasics.java`:

```
findUser(1).map(User::email).orElse("no email")
findUser(1).filter(u -> u.name().startsWith("a")).map(User::name).orElse("no match")
findUser(9).or(() -> findUser(1))           // an alternative Optional, so it chains
```

`ifPresentOrElse` handles the two-branch case. `Optional.stream()` yields zero or one element, so `flatMap(Optional::stream)` drops the misses inside a pipeline without a separate filter.

map versus flatMap

Same rule as streams. If your function returns a plain value, use `map`. If it already returns an `Optional`, `map` gives you a nested one:

```
map with an Optional-returning fn: Optional[Optional[ada@example.com]]
flatMap:                           Optional[ada@example.com]
```

When a chain suddenly has nested `Optional`s in it, you wanted `flatMap`.

orElse versus orElseGet

The single most common `Optional` bug. From `OrElseVsOrElseGet.java`, both calls made on a **present** `Optional`:

```
calling orElse:
>>> expensiveDefault ran, called from orElse
result: the real value
calling orElseGet:
result: the real value
```

Both returned the real value. Only one computed the default it then threw away.

	Takes	Evaluated
<code>orElse(T other)</code>	a value	always, because Java evaluates arguments before the call
<code>orElseGet(Supplier<T>)</code>	a lambda	only when empty

It stops being a performance note the moment the default has side effects:

```
after orElse on a PRESENT optional, counter = 1
after orElseGet on the same,               counter = 0
```

Now it is a correctness bug. A default that inserts a row, sends a request or logs will do so when it should not.

The rule: `orElse("")` and `orElse(0)` are fine. `orElse(buildDefault())` and `orElse(repo.findDefault())` are bugs.

`orElseThrow(supplier)` is the honest way to say "absent here is a bug", and it lets you write a message explaining what was expected.

Five anti-patterns

From `AntiPatterns.java`:

1. **`isPresent()` then `get()`**. This is an if-null check with more typing. Nearly every such pair becomes `map`, `filter`, `ifPresent`, `ifPresentOrElse`, `orElse` or `orElseGet`.
2. **`Optional` as a field**. It is not `Serializable`, it costs an object per instance, and a field can simply be null. Keep the field plain and return an `Optional` from the **accessor**.
3. **`Optional` as a parameter**. Every caller must now write `Optional.of(x)` or `Optional.empty()`, and they can still pass null, so you gained nothing. Use an overload.
4. **`Optional<List<T>>`**. The caller now has three cases: absent, present-and-empty, present-and-populated. An empty list already means nothing here.
5. **`Optional` in a hot loop**. Every one is an allocation. This is one of the few places where that argument genuinely holds.

The crash it was meant to prevent

`UnguardedGet.java`:

```
Exception in thread "main" java.util.NoSuchElementException: No value present
```

This is worse than a `NullPointerException`, not better. Since Java 14 an NPE names the exact expression that was null. "No value present" names nothing. And the code *looks* safe, because `Optional` appears in the type, so a reviewer skims past it.

`get()` has no use that `orElseThrow()` does not cover better. JDK 10 added the no-argument `orElseThrow()` for exactly this reason: identical behaviour, but it reads as a deliberate choice rather than an oversight.

Run It

```
java modules/15-optional/examples/OptionalBasics.java
java modules/15-optional/examples/OrElseVsOrElseGet.java
java modules/15-optional/examples/AntiPatterns.java

# Fails on purpose.
java modules/15-optional/examples/UnguardedGet.java

./scripts/verify-examples.sh modules/15-optional
```

Common Mistakes

`orElse` with a computed default. It runs every time, present or not.

`isPresent()` followed by `get()`. You have written a null check wearing a costume.

`Optional` fields and parameters. Neither was the intent, and both add noise.

Returning `Optional<List<T>>`. Return an empty list.

Calling `get()` at all. Use `orElseThrow` with a message.

Returning `null` from a method that returns `Optional`. It happens, and it is the worst of both worlds: the caller trusts the type and gets an NPE anyway.

Key Takeaways

- `Optional` is a return type for lookups, not a general null wrapper.
- It is a convention, not a guarantee. Java's compiler does not enforce it.
- Describe what to do, never ask whether it is there.
- `flatMap` when the function already returns an `Optional`.
- `orElse` always evaluates its argument; `orElseGet` does not. With side effects, that is a correctness bug.
- Never `get()`. `orElseThrow` with a message says what you expected.

Homework

[homework/README.md](#)

Build a lookup chain, prove the `orElse` trap with a counter, and fix four anti-patterns. Reference solution in [solutions/15-optional/](#).